

MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY

School of Computing · LY-IT-CORE

Order Management System

Spring Boot · JPA · Hibernate · REST API

Sahil S. Jirapure

ADT23SOCB1630

Academic Year 2025 – 26

1. Overview

A RESTful web application built with Spring Boot that manages the full lifecycle of customers, products, and orders. The system implements layered architecture, JPA entity relationships, and custom query methods, serving as a practical demonstration of enterprise backend development patterns.

2. Technologies

Technology	Purpose
Java 21	Core programming language
Spring Boot 3	Application framework & embedded Tomcat
Spring Data JPA	Repository abstraction and query derivation
Hibernate	ORM engine — entity ↔ table mapping
H2 Database	In-memory relational database
Lombok	Boilerplate elimination (@Data, @Getter, @Setter)
Maven	Build tool and dependency management
Postman	REST API testing and verification

3. Architecture

Standard three-layer Spring MVC structure:

Layer	Classes	Responsibility
Controller	CustomerController, OrderController, ProductController	Route HTTP requests
Service	CustomerService, OrderService, ProductService	Business logic
Repository	CustomerRepository, OrderRepository, ProductRepository	Data access via JPA
Entity	Customer, Product, Orders	Database-mapped objects
DTO	CustomerDto	Data transfer / projection

4. Entity Design

Customer (table: SwiggyCustomers)

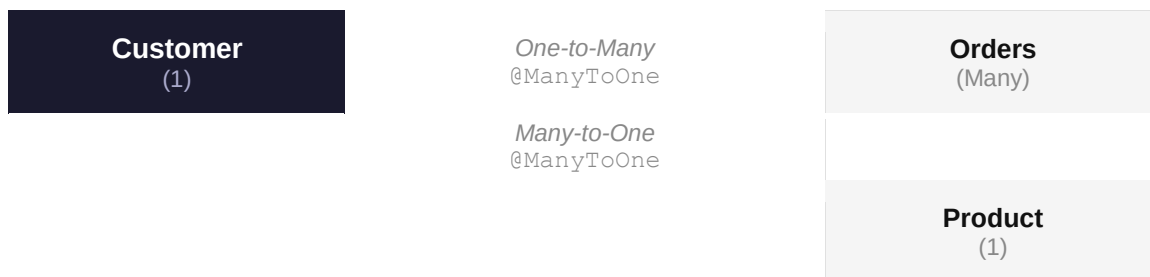
Field	Type	Constraint
customerId	Long	PK · Auto-generated
name	String	NOT NULL
email	String	UNIQUE
city	String	—
age	int	—
bankUserName	String	—
password	int	—

Product (table: product)

Field	Type	Constraint
productId	Long	PK · Auto-generated
productName	String	—
price	double	—
category	String	—
quantity	int	—

Orders (table: Orders)

Field	Type	Notes
orderId	Long	PK · Auto-generated
quantityOrdered	int	—
totalPrice	double	price × qty (computed in service)
customer	Customer	FK → SwiggyCustomers (@ManyToOne)
product	Product	FK → product (@ManyToOne)

Relationship

5. API Endpoints

Customer /customers

Method	Endpoint	Description
GET	/customers	Get all customers
GET	/customers/{id}	Get customer by ID
GET	/customers/email/{email}	Get by email
GET	/customers/city/{city}	Get by city
GET	/customers/age/{age}	Get where age > value
POST	/customers	Create customer
POST	/customers/list	Bulk create customers
PUT	/customers/{id}	Update customer
DELETE	/customers/{id}	Delete customer

Product /products

Method	Endpoint	Description
GET	/products	Get all products
GET	/products/{id}	Get product by ID
GET	/products/category/{cat}	Get by category
POST	/products	Add product
POST	/products/list	Bulk add products
PUT	/products/{id}	Update product
DELETE	/products/{id}	Delete product

Orders /orders

Method	Endpoint	Description
GET	/orders	Get all orders
POST	/orders/{custId}/{prodId}/{qty}	Place order · total = price × qty

6. API Testing — Postman Screenshots

All endpoints tested via Postman at `http://localhost:8082`. Responses verified 200 OK with correct JSON structure.

GET /customers/1

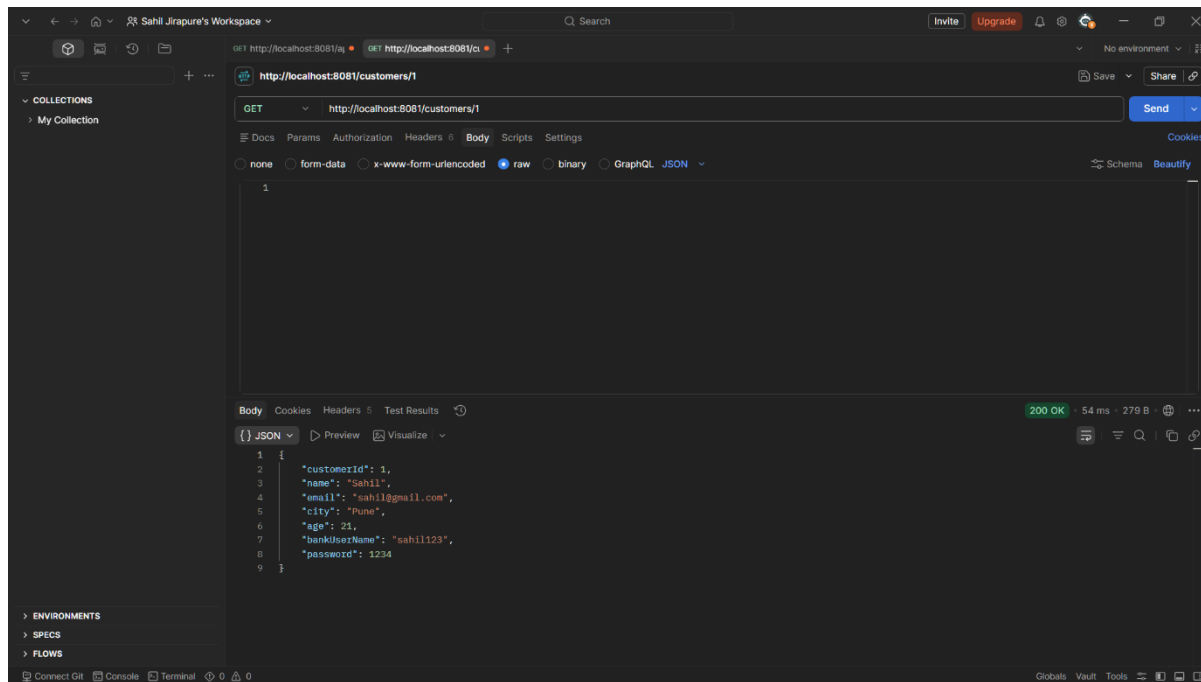


Figure 1 · GET /customers/1 — returns customer 1 records (200 OK, 16ms)

Post/customers/list

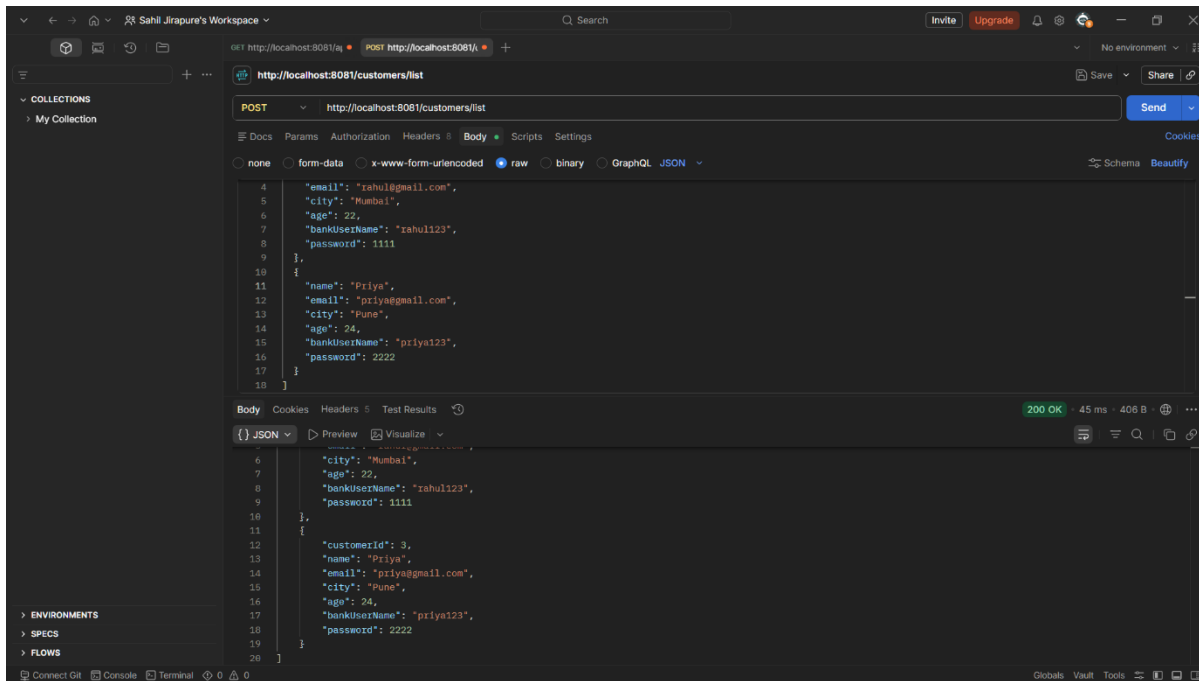


Figure 2 · POST /customers/list —add list of customer (200 OK, 16ms)

GET /products

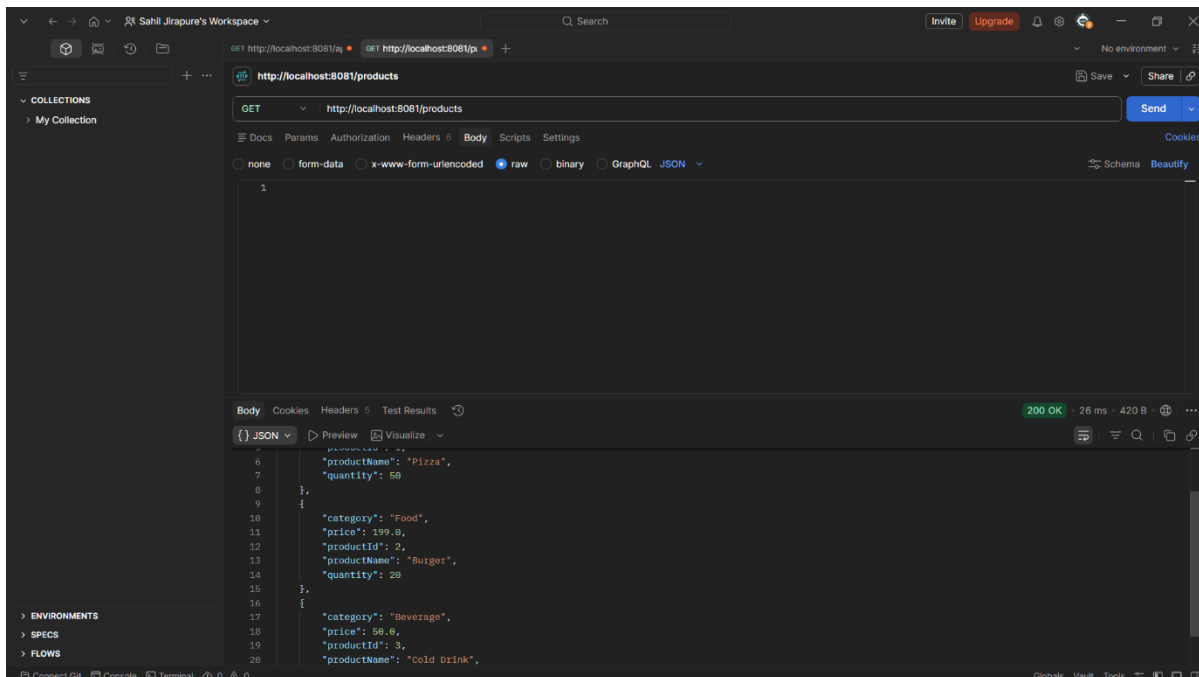


Figure 3 · GET /products — returns all product records (200 OK, 103ms)

GET /orders

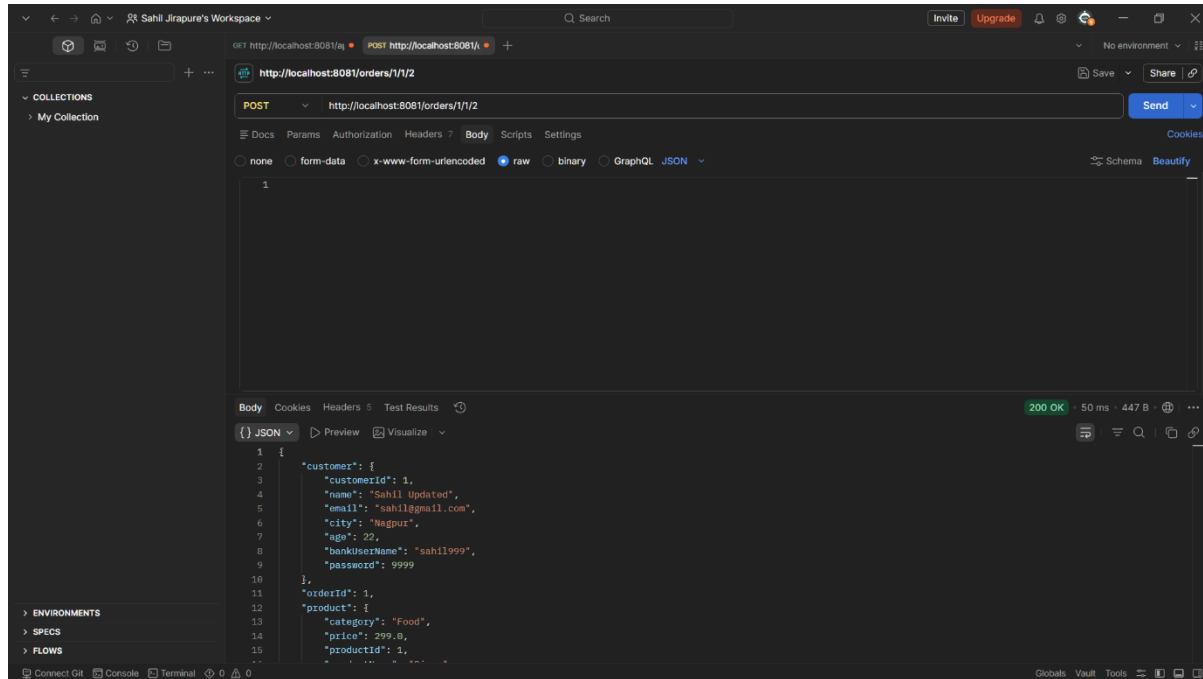


Figure 4 · `GET /orders` — returns full order objects with nested customer & product (200 OK, 19ms)

POST /products

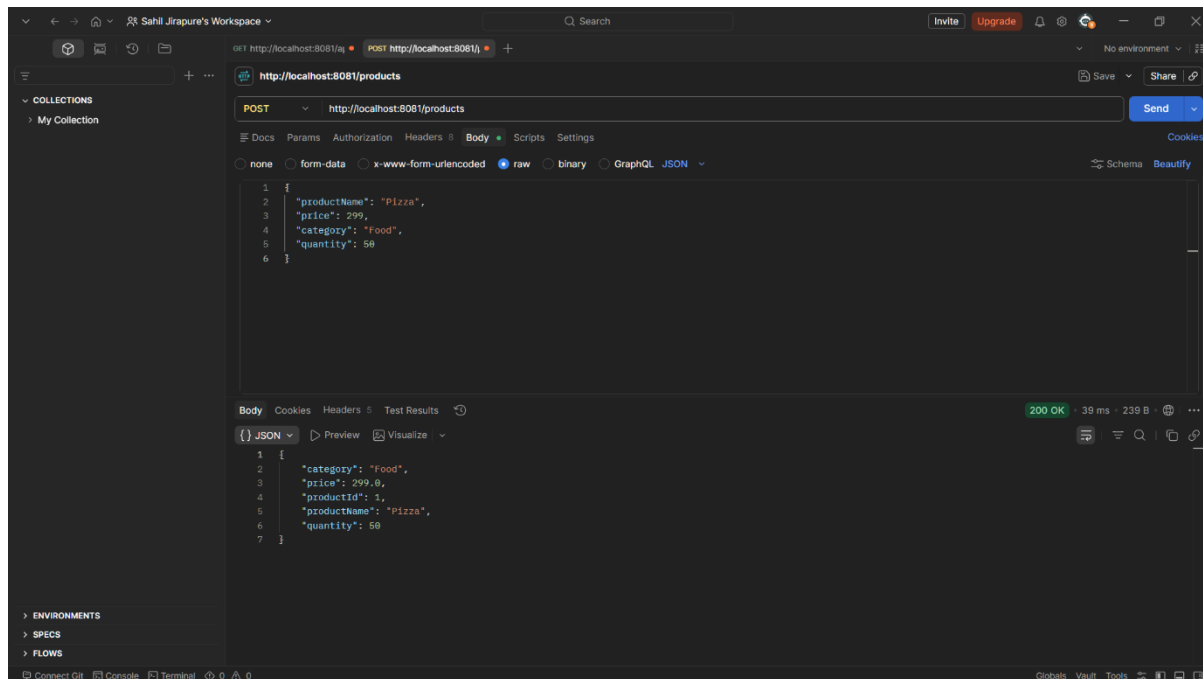


Figure 5 · `POST /products` — adds product (200 OK, 19ms)

DELETE/customers/3

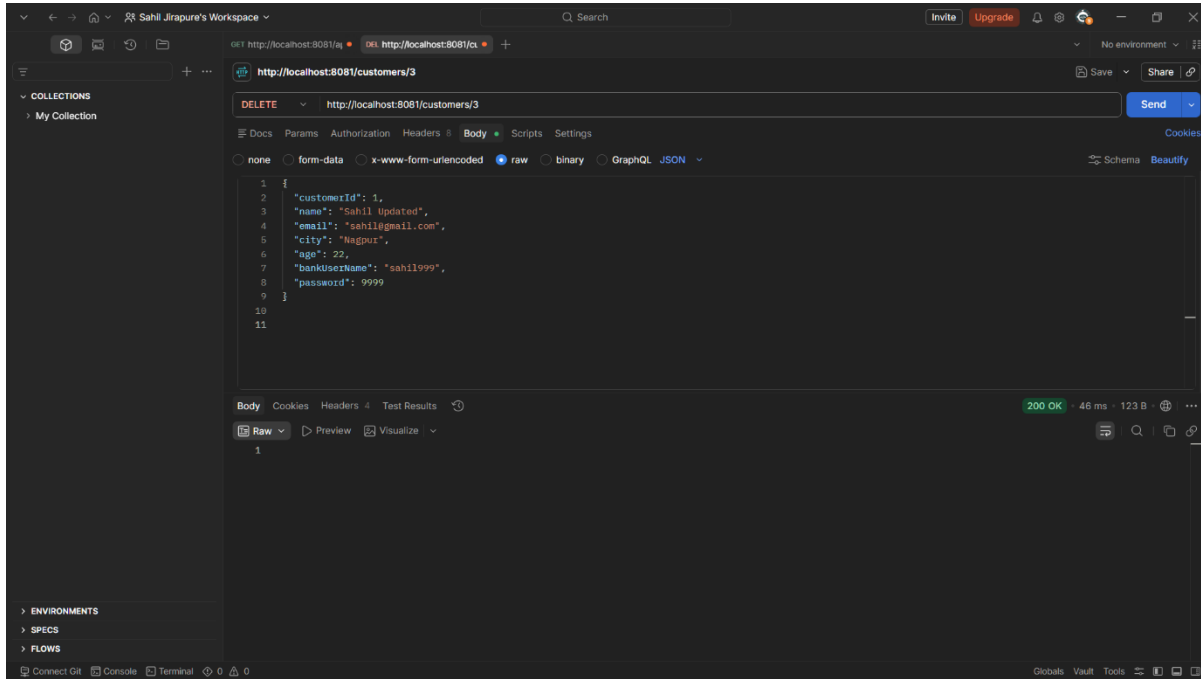
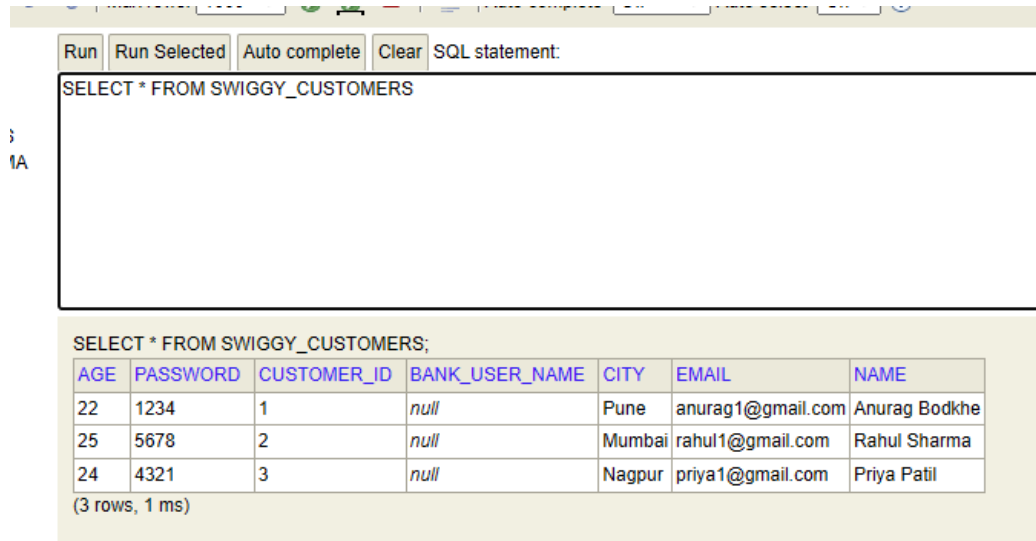


Figure 6 · DELETE/customers/3 — deletes customer 3 (200 OK, 19ms)

7. Database Verification — H2 Console

Data persistence verified using the H2 in-memory console at <http://localhost:8082/h2-console>.

SWIGGY_CUSTOMERS table



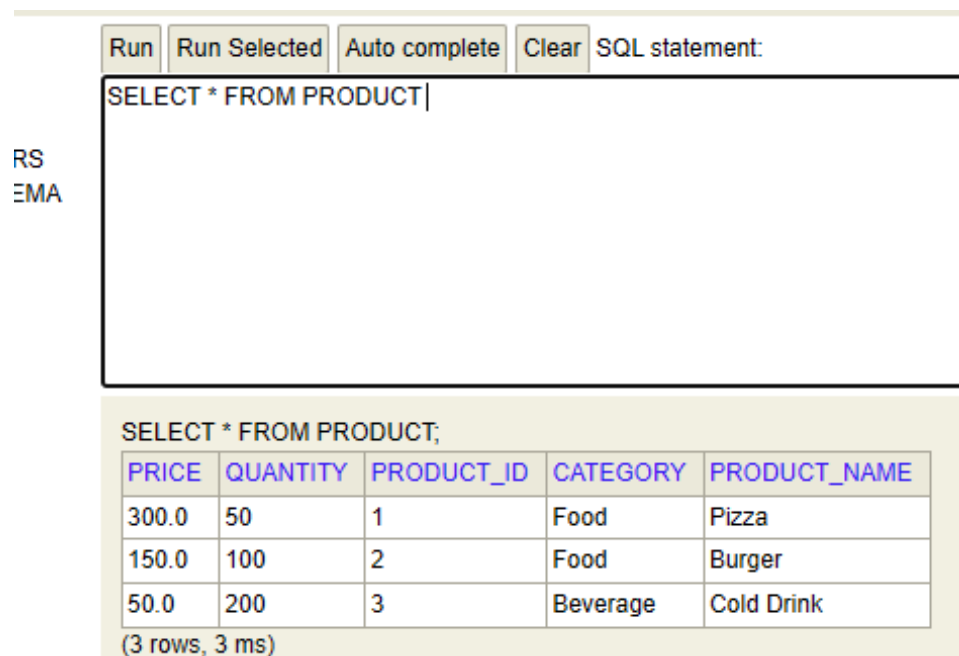
The screenshot shows the H2 Console interface. At the top, there are buttons: Run, Run Selected, Auto complete, and Clear. To the right of these buttons is a text input field labeled 'SQL statement:'. Below this, the SQL statement 'SELECT * FROM SWIGGY_CUSTOMERS' is entered. The console output shows the result of the query as a table with 7 columns: AGE, PASSWORD, CUSTOMER_ID, BANK_USER_NAME, CITY, EMAIL, and NAME. There are 3 rows of data. Below the table, it says '(3 rows, 1 ms)'.

AGE	PASSWORD	CUSTOMER_ID	BANK_USER_NAME	CITY	EMAIL	NAME
22	1234	1	null	Pune	anurag1@gmail.com	Anurag Bodkhe
25	5678	2	null	Mumbai	rahul1@gmail.com	Rahul Sharma
24	4321	3	null	Nagpur	priya1@gmail.com	Priya Patil

(3 rows, 1 ms)

Figure 4 · `SELECT * FROM SWIGGY_CUSTOMERS` — 3 rows persisted

PRODUCT table



The screenshot shows the H2 Console interface. At the top, there are buttons: Run, Run Selected, Auto complete, and Clear. To the right of these buttons is a text input field labeled 'SQL statement:'. Below this, the SQL statement 'SELECT * FROM PRODUCT' is entered. The console output shows the result of the query as a table with 5 columns: PRICE, QUANTITY, PRODUCT_ID, CATEGORY, and PRODUCT_NAME. There are 3 rows of data. Below the table, it says '(3 rows, 3 ms)'.

PRICE	QUANTITY	PRODUCT_ID	CATEGORY	PRODUCT_NAME
300.0	50	1	Food	Pizza
150.0	100	2	Food	Burger
50.0	200	3	Beverage	Cold Drink

(3 rows, 3 ms)

Figure 5 · `SELECT * FROM PRODUCT` — 3 products confirmed

ORDERS table

A

Run Run Selected Auto complete Clear SQL statement:

```
SELECT * FROM ORDERS|
```

SELECT * FROM ORDERS;

QUANTITY_ORDERED	TOTAL_PRICE	CUSTOMER_ID	ORDER_ID	PRODUCT_ID
2	600.0	1	1	1
3	450.0	2	2	2
5	250.0	3	3	3

(3 rows, 1 ms)

Figure 6 · *SELECT * FROM ORDERS* — foreign keys and totals verified

8. Repository — Custom Queries

Method	Repository	Type
findByEmail(email)	CustomerRepository	Derived query
findByCity(city)	CustomerRepository	Derived query
findByAgeGreaterThanOrEqual(age)	CustomerRepository	Derived query
findByCategory(category)	ProductRepository	Derived query
countOrder(customerId)	OrderRepository	@Query — JPQL
totalAmount(customerId)	OrderRepository	@Query — JPQL
totalRevenue()	OrderRepository	@Query — JPQL

9. Conclusion

The Swiggy Order Management System demonstrates a complete Spring Boot backend stack:

- Layered architecture with clean separation of Controller, Service, and Repository
- @ManyToOne JPA mappings maintaining relational integrity across three entities
- Mix of derived query methods and JPQL @Query for flexible data access
- Business logic (total price computation) contained in the service layer
- All endpoints verified with Postman; data confirmed via H2 console queries

Sahil Jirpure · ADT23SOCB1630 · LY-IT-CORE
MIT Art, Design and Technology University, Pune · 2024–25